

Практическое занятие 2. Функциональное тестирование программного обеспечения

В этой главе проходит обсуждение вопросов тестирования ПО на уровне программных модулей. Впрочем, рассматриваемое здесь функциональное тестирование, основанное на принципе «черного ящика», может применяться и на уровне программной системы. После определения особенностей тестирования «черного ящика» в главе описываются популярные способы тестирования: разбиение по классам эквивалентности, анализ граничных значений, тестирование на основе диаграмм причин–следствий.

Особенности тестирования «черного ящика»

Тестирование «черного ящика» (функциональное тестирование) позволяет получить комбинации входных данных, обеспечивающих полную проверку всех функциональных требований к программе. Программное изделие здесь рассматривается как «черный ящик», чье поведение можно определить только исследованием его входов и соответствующих выходов. При таком подходе желательно иметь:

- ❑ набор, образуемый такими входными данными, которые приводят к аномалиям поведения программы (назовем его *ИТ*);
 - ❑ набор, образуемый такими выходными данными, которые демонстрируют дефекты программы (назовем его *ОТ*).
- Как показано на рис. 1, любой способ тестирования «черного ящика» должен:
- ❑ выявить такие входные данные, которые с высокой вероятностью принадлежат набору *ИТ*;
 - ❑ сформулировать такие ожидаемые результаты, которые с высокой вероятностью являются элементами набора *ОТ*.

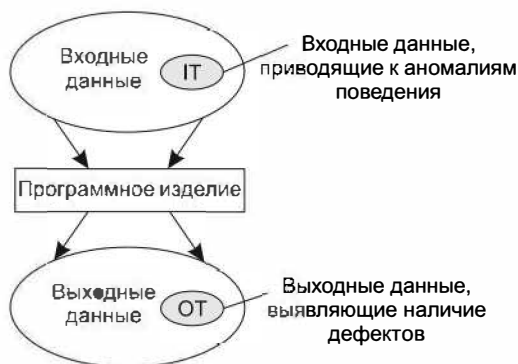


Рис. 1. Тестирование «черного ящика»

Во многих случаях определение таких тестовых вариантов основывается на предыдущем опыте инженеров тестирования. Они используют свое знание и понимание области определения для идентификации тестовых вариантов, которые эффективно обнаруживают дефекты. Тем не менее систематический подход к выявлению тестовых данных, обсуждаемый в данной главе, может использоваться как полезное дополнение к эвристическому знанию.

Принцип «черного ящика» не альтернативен принципу «белого ящика». Скорее это дополняющий подход, который обнаруживает другой класс ошибок.

Тестирование «черного ящика» выявляет следующие категории ошибок:

- 1) некорректные или отсутствующие функции;
- 2) ошибки интерфейса;
- 3) ошибки во внешних структурах данных или в доступе к внешней базе данных;
- 4) ошибки характеристик (необходимая емкость памяти и т. д.);
- 5) ошибки инициализации и завершения.

Подобные категории ошибок способами «белого ящика» не обнаруживаются.

В отличие от тестирования «белого ящика», которое выполняется на ранней стадии процесса тестирования, тестирование «черного ящика» применяют на поздних стадиях тестирования. При тестировании «черного ящика» пренебрегают управляющей структурой программы. Здесь внимание концентрируется на информационной области определения программной системы.

Техника «черного ящика» ориентирована на решение следующих задач:

- ☐ сокращение необходимого количества тестовых вариантов (из-за проверки не статических, а динамических аспектов системы);
- ☐ выявление классов ошибок, а не отдельных ошибок.

Способ разбиения по эквивалентности

Разбиение по эквивалентности — самый популярный способ тестирования «черного ящика».

В этом способе входная область данных программы делится на классы эквивалентности. Для каждого класса эквивалентности разрабатывается один тестовый вариант.

Класс эквивалентности — это набор данных с общими свойствами. Обработывая разные элементы класса, программа должна вести себя одинаково. Иначе говоря, при обработке любого набора из класса эквивалентности в программе задействуется один и тот же набор операторов (и связей между ними).

На рис. 15.2 каждый класс эквивалентности показан как эллипс. Здесь выделены входные классы эквивалентности допустимых и недопустимых исходных данных, а также классы результатов.

Классы эквивалентности могут быть определены по спецификации на программу.

Классы эквивалентности исходных данных

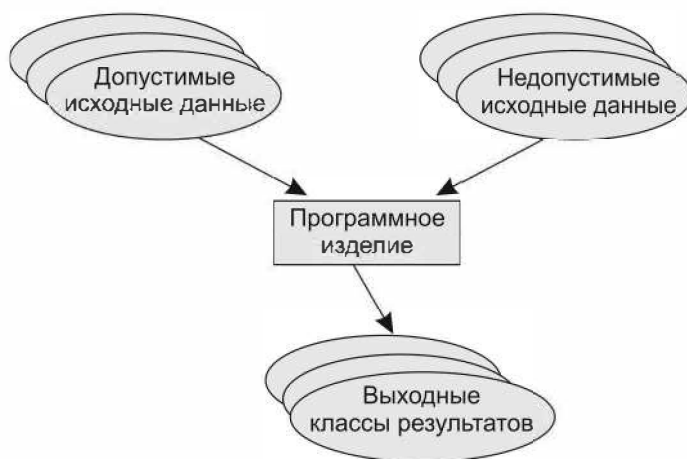


Рис. 2. Разбиение по эквивалентности

Например, если спецификация задает в качестве допустимых входных величин 5-разрядные целые числа в диапазоне 15 000 ... 70 000, то класс эквивалентности допустимых ИД (исходных данных) включает величины от 15 000 до 70 000, а два класса эквивалентности недопустимых ИД составляют:

- ❑ числа меньшие, чем 15 000;
- ❑ числа большие, чем 70 000.

Класс эквивалентности включает множество значений данных, допустимых или недопустимых по условиям ввода.

Условие ввода может задавать:

- 1) определенное значение;
- 2) диапазон значений;
- 3) множество конкретных величин;
- 4) булево условие.

Сформулируем *правила формирования классов эквивалентности*.

1. Если условие ввода задает диапазон $n...m$, то определяется один допустимый и два недопустимых класса эквивалентности:
 - $V_Class = \{n...m\}$ — допустимый класс эквивалентности;
 - $Inv_Class1 = \{x \mid \text{для любого } x: x < n\}$ — первый недопустимый класс эквивалентности;
 - $Inv_Class2 = \{y \mid \text{для любого } y: y > m\}$ — второй недопустимый класс эквивалентности.
2. Если условие ввода задает конкретное значение a , то определяется один допустимый и два недопустимых класса эквивалентности:
 - $V_Class = \{a\}$;
 - $Inv_Class1 = \{x \mid \text{для любого } x: x < a\}$;
 - $Inv_Class2 = \{y \mid \text{для любого } y: y > a\}$.
3. Если условие ввода задает множество значений $\{a, b, c\}$, то определяется один допустимый и один недопустимый класс эквивалентности:
 - $V_Class = \{a, b, c\}$;
 - $Inv_Class = \{x \mid \text{для любого } x: (x \neq a) \& (x \neq b) \& (x \neq c)\}$.
4. Если условие ввода задает булево значение, например *true*, то определяется один допустимый и один недопустимый класс эквивалентности:
 - $V_Class = \{true\}$;
 - $Inv_Class = \{false\}$.

После построения классов эквивалентности разрабатываются тестовые варианты. Тестовый вариант выбирается так, чтобы проверить сразу наибольшее количество свойств класса эквивалентности.

Способ анализа граничных значений

Как правило, большая часть ошибок происходит на границах области ввода, а не в центре. Анализ граничных значений заключается в получении тестовых вариантов, которые анализируют граничные значения. Данный способ тестирования дополняет способ разбиения по эквивалентности.

Основные отличия анализа граничных значений от разбиения по эквивалентности:

- 1) тестовые варианты создаются для проверки только ребер классов эквивалентности;
- 2) при создании тестовых вариантов учитывают не только условия ввода, но и область вывода.

Сформулируем *правила анализа граничных значений*:

1. Если условие ввода задает диапазон $n...m$, то тестовые варианты должны быть построены:
 - для значений n и m ;
 - для значений чуть левее n и чуть правее m на числовой оси.

Например, если задать входной диапазон $-1,0 \dots +1,0$, то создаются тесты для значений $-1,0, +1,0, -1,001, +1,001$.

2. Если условие ввода задает дискретное множество значений, то создаются тестовые варианты:

- для проверки минимального и максимального из значений;
- для значений чуть меньше минимума и чуть больше максимума.

Так, если входной файл может содержать от 1 до 255 записей, то создаются тесты для 0, 1, 255, 256 записей.

3. Правила 1 и 2 применяются к условиям области вывода.

Рассмотрим пример, когда в программе требуется выводить таблицу значений. Количество строк и столбцов в таблице меняется. Задается тестовый вариант для минимального вывода (по объему таблицы), а также тестовый вариант для максимального вывода (по объему таблицы).

4. Если внутренние структуры данных программы имеют предписанные границы, то разрабатываются тестовые варианты, проверяющие эти структуры на их границах.
5. Если входные или выходные данные программы являются упорядоченными множествами (например, последовательным файлом, линейным списком, таблицей), то надо тестировать обработку первого и последнего элементов этих множеств.

Большинство разработчиков используют этот способ интуитивно. При применении описанных правил тестирование границ будет более полным, в связи с чем возрастет вероятность обнаружения ошибок.

Пример 1. Рассмотрим применение способов разбиения по эквивалентности и анализа граничных значений на конкретном примере. Положим, что нужно протестировать программу бинарного поиска. Нам известна *спецификация* этой программы. Поиск выполняется в массиве элементов M , возвращается индекс I элемента массива, значение которого соответствует ключу поиска Key .

Предусловия:

- 1) массив должен быть упорядочен;
- 2) массив должен иметь не менее одного элемента;
- 3) нижняя граница массива (индекс) должна быть меньше или равна его верхней границе.

Постусловия:

- 1) если элемент найден, то флаг **Result=True**, значение I — номер элемента;
- 2) если элемент не найден, то флаг **Result=False**, значение I не определено.

Для формирования классов эквивалентности (и их ребер) надо произвести разбиение области ИД — построить дерево разбиений. Листья дерева разбиений дадут нам искомые классы эквивалентности. Определим стратегию разбиения. На первом уровне будем анализировать выполнимость предусловий, на втором уровне — выполнимость постусловий. На третьем уровне можно анализировать специальные требования, полученные из практики разработчика. В нашем примере

мы знаем, что входной массив должен быть упорядочен. Обработка упорядоченных наборов из четного и нечетного количества элементов может выполняться по-разному. Кроме того, принято выделять специальный случай одноэлементного массива. Следовательно, на уровне специальных требований возможны следующие эквивалентные разбиения:

- 1) массив из одного элемента;
- 2) массив из четного количества элементов;
- 3) массив из нечетного количества элементов, большего единицы.

Наконец, на последнем, четвертом уровне критерием разбиения может быть анализ ребер классов эквивалентности. Очевидно, возможны следующие варианты:

- 1) работа с первым элементом массива;
- 2) работа с последним элементом массива;
- 3) работа с промежуточным (ни с первым, ни с последним) элементом массива.

Структура дерева разбиений приведена на рис. 3.

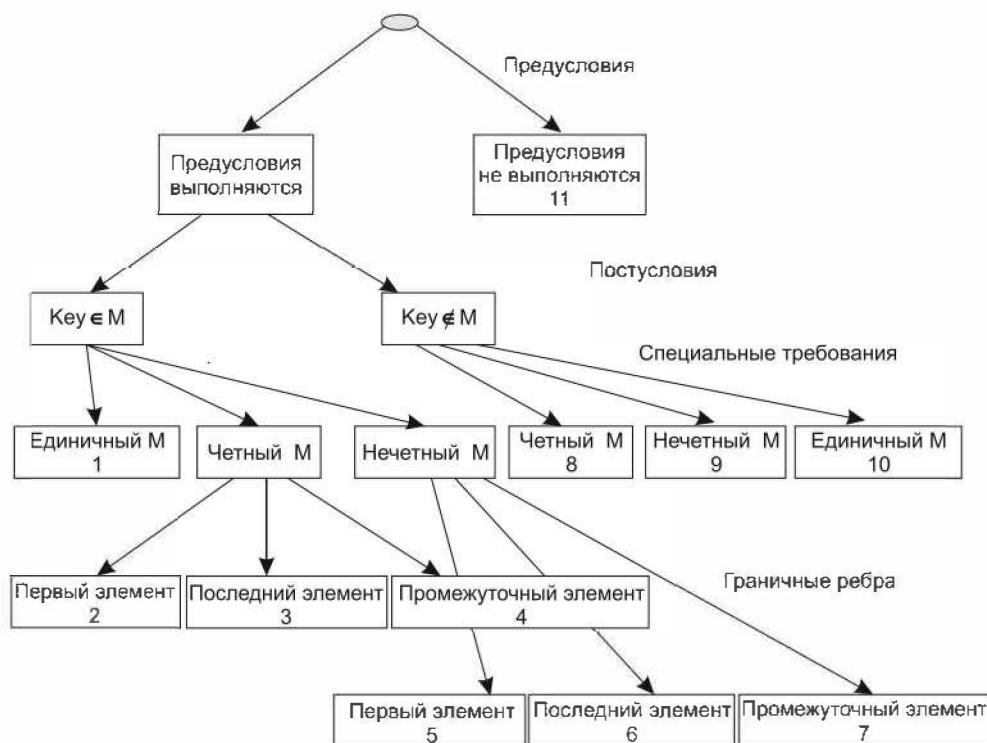


Рис. 3. Дерево разбиений области исходных данных бинарного поиска

Это дерево имеет 11 листьев. Каждый лист задает отдельный тестовый вариант. Покажем тестовые варианты, основанные на проведенных разбиениях.

Тестовый вариант 1 (единичный массив, элемент найден) ТВ1:

ИД: М = 15; Key = 15.

ОЖ.РЕЗ.: Result=True; I = 1.

Тестовый вариант 2 (четный массив, найден 1-й элемент) **ТВ2:**

ИД: М = 15, 20, 25, 30, 35, 40; Key = 15.

ОЖ.РЕЗ.: Result=True; I = 1.

Тестовый вариант 3 (четный массив, найден последний элемент) **ТВ3:**

ИД: М = 15, 20, 25, 30, 35, 40; Key = 40.

ОЖ.РЕЗ.: Result=True; I = 6.

Тестовый вариант 4 (четный массив, найден промежуточный элемент) **ТВ4:**

ИД: М = 15, 20, 25, 30, 35, 40; Key = 25.

ОЖ.РЕЗ.: Result=True; I = 3.

Тестовый вариант 5 (нечетный массив, найден 1-й элемент) **ТВ5:**

ИД: М = 15, 20, 25, 30, 35, 40, 45; Key = 15.

ОЖ.РЕЗ.: Result=True; I = 1.

Тестовый вариант 6 (нечетный массив, найден последний элемент) **ТВ6:**

ИД: М = 15, 20, 25, 30, 35, 40, 45; Key = 45.

ОЖ.РЕЗ.: Result=True; I = 15.

Тестовый вариант 7 (нечетный массив, найден промежуточный элемент) **ТВ7:**

ИД: М = 15, 20, 25, 30, 35, 40, 45; Key = 30.

ОЖ.РЕЗ.: Result=True; I = 4.

Тестовый вариант 8 (четный массив, не найден элемент) **ТВ8:**

ИД: М = 15, 20, 25, 30, 35, 40; Key = 23.

ОЖ.РЕЗ.: Result=False; I = ?

Тестовый вариант 9 (нечетный массив, не найден элемент) **ТВ9:**

ИД: М = 15, 20, 25, 30, 35, 40, 45; Key = 24.

ОЖ.РЕЗ.: Result=False; I = ?

Тестовый вариант 10 (единичный массив, не найден элемент) **ТВ10:**

ИД: М = 15; Key = 0.

ОЖ.РЕЗ.: Result=False; I = ?

Тестовый вариант 11 (нарушены предусловия) **ТВ11:**

ИД: М = 15, 10, 5, 25, 20, 40, 35; Key = 35.

ОЖ.РЕЗ.: Аварийное донесение: Массив не упорядочен.

Пример 2. Протестируем операцию покупки товара в торговом автомате за наличные деньги.

Предусловия операции:

- ☐ покупатель вносит деньги;
- ☐ количество запрашиваемого товара введено правильно.

Постусловия операции:

- ☐ если нет проблем с деньгами и товаром, то выдается запрошенное количество товара;
- ☐ если есть проблемы с деньгами или товаром, то они устраняются.

Для формирования классов эквивалентности применим четырехуровневую стратегию разбиения области исходных данных: на первом уровне работаем с предусловиями, на втором — с постусловиями, на третьем — со специальными требованиями, на четвертом — с граничными значениями.

Специальные требования применяются при наличии проблемы постусловий. Они дают два варианта:

- 1) источник проблемы — деньги;
- 2) источник проблемы — товар.

Уровень граничных ребер выделяет три варианта:

- 1) внесена избыточная сумма денег;
- 2) внесена недостаточная сумма денег;
- 3) нет необходимого количества товара.

Структура дерева разбиений показана на рис. 4.

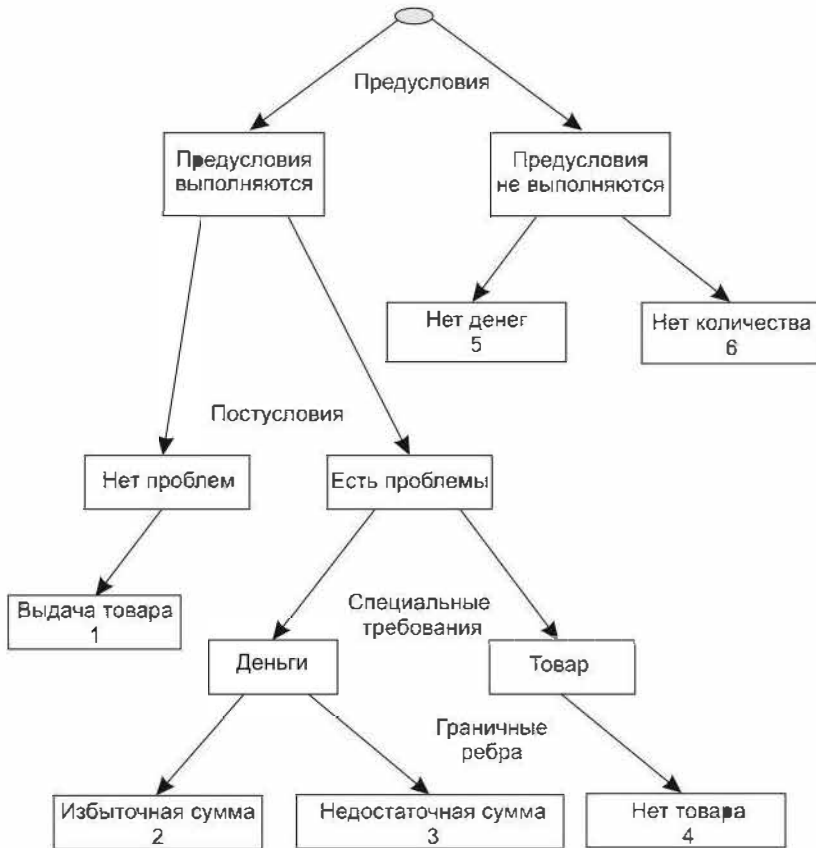


Рис. 4. Дерево разбиений области исходных данных для операции покупки товара

Данное дерево имеет шесть листьев, каждый из которых соответствует отдельному тестовому варианту.

Покажем тестовые варианты, основанные на проведенных разбиениях.

Тестовый вариант 1 (нет проблем, выдача товара) ТВ1:

ИД: Количество приобретаемого товара введено корректно и не превышает количества товара, имеющегося в автомате. Покупатель вносит сумму, равную стоимости покупки.

ОЖ.РЕЗ.: Выдача запрошенного количества товара.

Тестовый вариант 2 (проблема — деньги, избыточная сумма) ТВ2:

ИД: Количество приобретаемого товара введено корректно и не превышает количества товара, имеющегося в автомате. Покупатель вносит сумму, превышающую стоимость покупки.

ОЖ.РЕЗ.: Выдача запрошенного количества товара и сдачи.

Тестовый вариант 3 (проблема — деньги, недостаточная сумма) ТВ3:

ИД: Количество приобретаемого товара введено корректно и не превышает количества товара, имеющегося в автомате. Покупатель вносит сумму, меньшую стоимости покупки.

ОЖ.РЕЗ.: Возврат вписанных в автомат денег и вывод сообщения «Не хватает денег».

Тестовый вариант 4 (проблема — товар, нет товара) ТВ4:

ИД: Количество приобретаемого товара введено корректно, но превышает количество товара, имеющегося в автомате. Покупатель вносит некоторую сумму денег.

ОЖ.РЕЗ.: Возврат вписанных в автомат денег и вывод сообщения «Не хватает продуктов».

Тестовый вариант 5 (нарушены предусловия, нет денег) ТВ5:

ИД: Количество приобретаемого товара введено корректно, но деньги не внесены.

ОЖ.РЕЗ.: Вывод сообщения «Не хватает денег».

Тестовый вариант 6 (нарушены предусловия, нет количества) ТВ6:

ИД: Количество приобретаемого товара введено некорректно (например, введено нулевое значение или символ, не являющийся цифрой).

ОЖ.РЕЗ.: Вывод сообщений «Ошибка ввода» и «Введите количество продуктов». Покупателю разрешаются три попытки ввода.

Способ диаграмм причин–следствий

Диаграммы причинно-следственных связей — способ проектирования тестовых вариантов, который обеспечивает формальную запись логических условий и соответствующих действий. Используется автоматный подход к решению задачи.

Шаги способа:

- 1) для каждого модуля перечисляются причины (условия ввода или классы эквивалентности условий ввода) и следствия (действия или условия вывода). Каждой причине и последствию присваивается свой идентификатор;
- 2) разрабатывается граф причинно-следственных связей;
- 3) граф преобразуется в таблицу решений;
- 4) столбцы таблицы решений преобразуются в тестовые варианты.

Изобразим базовые символы для записи графов причин и следствий (cause-effect graphs).

Сделаем предварительные *замечания*:

- 1) причины будем обозначать символами c_i , а следствия — символами e_i ;
- 2) каждый узел графа может находиться в состоянии 0 или 1 (0 — состояние отсутствует, 1 — состояние присутствует).

Функция тождество (рис. 5) устанавливает, что если значение c_1 есть 1, то и значение e_1 есть 1; в противном случае значение e_1 есть 0.

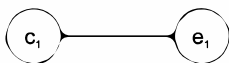


Рис. 5. Функция тождество

Функция не (рис. 6) устанавливает, что если значение c_1 есть 1, то значение e_1 есть 0; в противном случае значение e_1 есть 1.

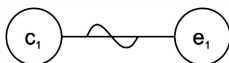


Рис. 6. Функция не

Функция или (рис. 7) устанавливает, что если c_1 или c_2 есть 1, то e_1 есть 1, в противном случае e_1 есть 0.

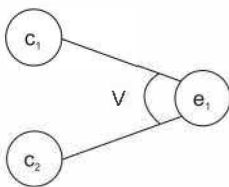


Рис. 7. Функция или

Функция и (рис. 8) устанавливает, что если и c_1 и c_2 есть 1, то e_1 есть 1, в противном случае e_1 есть 0.

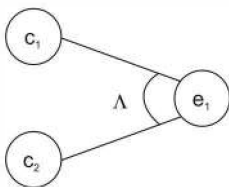


Рис. 8. Функция и

Часто определенные комбинации причин невозможны из-за синтаксических или внешних ограничений. Используются неречисленные ниже обозначения ограничений.

Ограничение *E* (исключает, Exclusive, рис. 9) устанавливает, что *E* должно быть истинным, если причины нулевые, или только одна из причин — *a* или *b* — при-

нимает значение 1 (*a* и *b* не могут припимать значение 1 одновременно, табл. 1). Иными словами, чтобы *E* сохрапяло истипное значепие, лишь одна из причин *может* припятть значение 1.

Таблица 1. Таблица истинности для ограничения E

Ограничение E	a	b
true	0	0
true	0	1
true	1	0
false	1	1

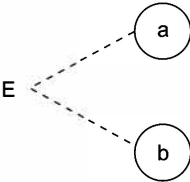


Рис. 9. Ограничение E (исключает, Exclusive)

Ограничение I (включает, Inclusive, рис.10) устанавливает, что по крайпей мере одна из величин *a*, *b* или *c* всегда должна быть равной 1 (*a*, *b* и *c* не могут при-пимать значение 0 одповреппо, табл. 2).

Таблица 2. Таблица истинности для ограничения I

Ограничение I	a	b	c
false	0	0	0
true	0	0	1
	0	1	0
	0	1	1
	1	0	0
	1	0	1
	1	1	0
	1	1	1

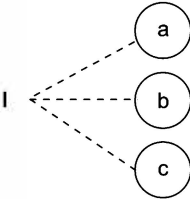
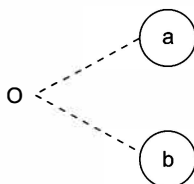


Рис.10. Ограничение I (включает, Inclusive)

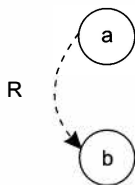
Ограничение O (одно и только одно, Only one, рис.11) устанавливает, что одна и только одна из величин *a* или *b* должна быть равна 1. Это показано в табл. 3.

Таблица 3. Таблица истинности для ограничения О

Ограничение О	a	b
false	0	0
true	0	1
true	1	0
false	1	1

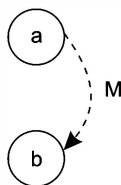
**Рис. 11.** Ограничение О (одно и только одно, Only one)

Ограничение *R* (требуется, *Requires*, рис. 12) устанавливает, что если *a* принимает значение 1, то и *b* должна принимать значение 1 (нельзя, чтобы *a* было равно 1, а *b* — 0).

**Рис. 12.** Ограничение R (требуется, *Requires*)

Часто возникает необходимость в ограничениях для следствий.

Ограничение *M* (скрывает, *Masks*, рис. 13) устанавливает, что если следствие *a* имеет значение 1, то следствие *b* должно принять значение 0.

**Рис. 13.** Ограничение M (скрывает, *Masks*)

Пример 3. Для иллюстрации использования способа рассмотрим пример, когда программа выполняет расчет оплаты за электричество по среднему или переменному тарифу.

При расчете по среднему тарифу:

- при месячном потреблении энергии меньшем, чем 100 кВт/ч, выставляется фиксированная сумма;

- при потреблении энергии большим или равном 100 кВт/ч применяется процедура *A* планирования расчета.

При расчете по переменному тарифу:

- при месячном потреблении энергии меньшем, чем 100 кВт/ч, применяется процедура *A* планирования расчета;
- при потреблении энергии большим или равном 100 кВт/ч применяется процедура *B* планирования расчета.

Шаг 1. Причинами являются:

- 1) расчет по среднему тарифу;
- 2) расчет по переменному тарифу;
- 3) месячное потребление электроэнергии меньшее, чем 100 кВт/ч;
- 4) месячное потребление электроэнергии большее или равное 100 кВт/ч.

На основе различных комбинаций причин можно перечислить следующие следствия:

- 101 — минимальная месячная стоимость;
- 102 — процедура *A* планирования расчета;
- 103 — процедура *B* планирования расчета.

Шаг 2. Разработка графа причинно-следственных связей (рис. 14).

Узлы причин перечислим по вертикали у левого края рисунка, а узлы следствий — у правого края рисунка. Для следствия 102 возникает необходимость введения вторичных причин — 11 и 12; их размещаем в центральной части рисунка.

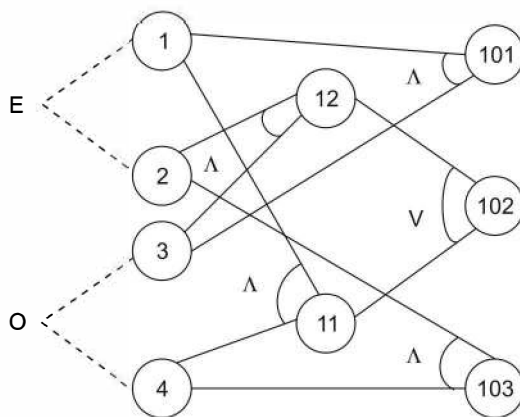


Рис. 14. Граф причинно-следственных связей

Шаг 3. Генерация таблицы решений. При генерации причины рассматриваются как условия, а следствия — как действия.

Порядок генерации:

1. Выбирается некоторое следствие, которое должно быть в состоянии «1».

2. Находятся все комбинации причин (с учетом ограничений), которые устанавливают это следствие в состояние «1». Для этого из следствия прокладывается обратная трасса через граф.
 3. Для каждой комбинации причин, приводящих следствие в состояние «1», строится один столбец.
 4. Для каждой комбинации причин доопределяются состояния всех других следствий. Они помещаются в тот же столбец таблицы решений.
 5. Действия 1–4 повторяются для всех следствий графа.
- Решения для нашего примера показаны в табл. 4.

Таблица 4. Таблица решений для расчета оплаты за электричество

Номера столбцов			1	2	3	4
Условия	Причины	1	1	0	1	0
		2	0	1	0	1
		3	1	1	0	0
		4	0	0	1	1
	Вторичные причины	11	0	0	1	0
		12	0	1	0	0
Действия	Следствия	101	1	0	0	0
		102	0	1	1	0
		103	0	0	0	1

Шаг 4. Преобразование каждого столбца таблицы в тестовый вариант. В нашем примере таких вариантов четыре.

Тестовый вариант 1 (столбец 1) ТВ1:

ИД: расчет по среднему тарифу; месячное потребление электроэнергии 75 кВт/ч.
ОЖ.РЕЗ.: минимальная месячная стоимость.

Тестовый вариант 2 (столбец 2) ТВ2:

ИД: расчет по неремонному тарифу; месячное потребление электроэнергии 90 кВт/ч.

ОЖ.РЕЗ.: процедура А напировапия расчета.

Тестовый вариант 3 (столбец 3) ТВ3:

ИД: расчет по среднему тарифу; месячное потребление электроэнергии 100 кВт/ч.

ОЖ.РЕЗ.: процедура А напировапия расчета.

Тестовый вариант 4 (столбец 4) ТВ4:

ИД: расчет по неремонному тарифу; месячное потребление электроэнергии 100 кВт/ч.

ОЖ.РЕЗ.: процедура В плапировапия расчета.

Контрольные вопросы и упражнения

1. Каковы особенности тестирования «черного ящика»?
2. Какие категории ошибок выявляет тестирование «черного ящика»?

3. Поясните суть способа разбиения на эквивалентности.
4. Какие правила формирования классов эквивалентности вы знаете?
5. Поясните суть способа анализа граничных значений.
6. Чем способ анализа граничных значений отличается от разбиения по эквивалентности?
7. Что такое дерево разбиений? Каковы его особенности?
8. В чем суть способа диаграмм причин–следствий?
9. Дайте общую характеристику графа причинно–следственных связей.
10. Поясните шаги способа диаграмм причин–следствий.
11. Какую структуру имеет таблица решений в способе диаграмм причин–следствий?
12. Как таблица решений преобразуется в тестовые варианты?